PRACTICAL 12

Deploy Nginx using Docker and Kubernetes

A hands-on lab for learning Docker containers and Kubernetes Deployments, Pods, and Services.

Student details

Field	Details
Name	Ashwini Birmal Burgute_____
Roll No. / ID	MCA2504004_____
Batch / Section	2025-27_____
Date performed	10 Sept., 2026_____

Learning objectives

By the end of this practical, you should be able to explain and demonstrate:

- What a Kubernetes cluster is
- What a Node is
- What a Pod is
- What a Deployment is
- What a Service is
- How Kubernetes uses Docker / container images
- How to deploy an application on Kubernetes
- How to scale an application
- How to check application logs
- How to delete Kubernetes resources

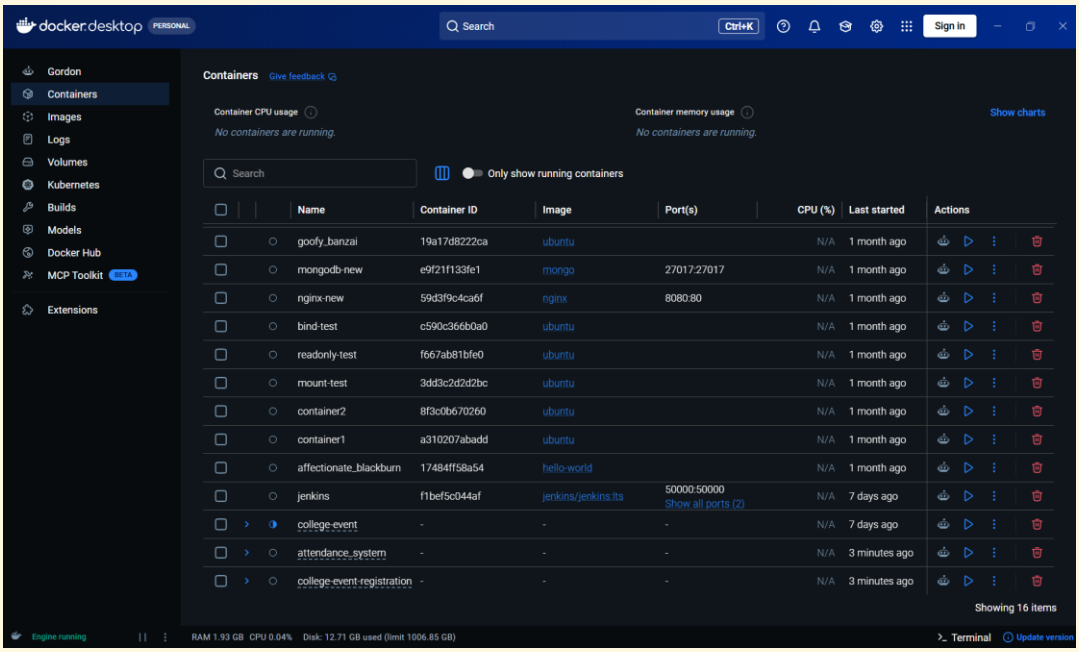
Before you start

Don't start with Kubernetes YAML, AKS, Helm, Ingress, Terraform, or Jenkins yet. This first practical is done entirely with docker and kubectl commands. Practical 2 will build your own image, push it to Docker Hub, and deploy it with a YAML Deployment and Service — that's where Docker and Kubernetes really connect.

Part A — Check Docker

Step A1: Start Docker Desktop

Open Docker Desktop and make sure it is running (the whale icon should be steady, not animating).



Step A2: Check the Docker version

Open Command Prompt or PowerShell and check that Docker is installed correctly.

Run:

```
docker --version
```

Expected output:

```
Docker version 28.x.x
```

Step A3: Check running containers

It is okay if the list is empty — you just want to confirm the command works.

Run:

```
docker ps
```

Expected output:

```
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

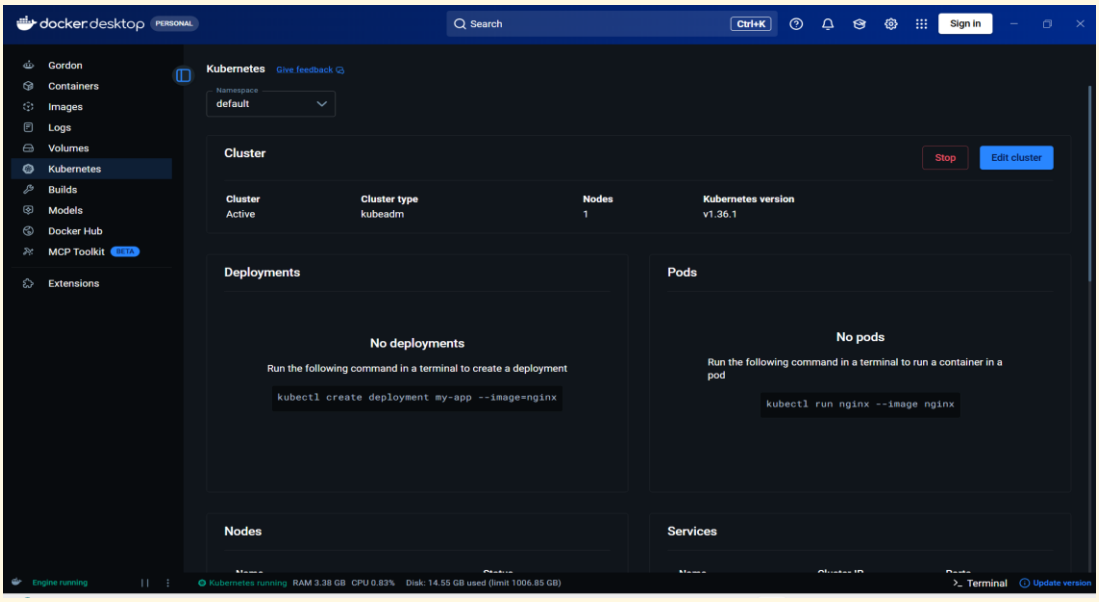
```
C:\>docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Part B — Enable Kubernetes

In a recent Docker Desktop version: go to Docker Desktop → Kubernetes → Create cluster. Docker Desktop supports local clusters using kubeadm or kind — for this practical, a single-node kubeadm cluster is enough.

In the older interface, instead go to Settings → Kubernetes → Enable Kubernetes, then let Docker Desktop start the cluster.

Wait until Docker Desktop shows the cluster as running before continuing.



Part C — Check Kubernetes

Step C1: Check the kubectl version

Run:

```
kubectl version
```

Step C2: Check cluster nodes

The important word to look for is Ready.

Run:

```
kubectl get nodes
```

Expected output:

```
NAME             STATUS   ROLES           AGE   VERSION
docker-desktop   Ready    control-plane   ...   v1.xx.x
```

```
C:\>kubectl get nodes
NAME             STATUS   ROLES           AGE   VERSION
docker-desktop   Ready    control-plane   12m   v1.36.1
```

If you see Ready, congratulations — you have your first working Kubernetes cluster.

Part D — Understand what you just created

Before doing the practical, understand the layers involved:

```
Your Computer
     |
     v
Docker Desktop
     |
     v
Kubernetes Cluster
     |
     v
     Node
     |
     v
     Pod
     |
     v
Nginx Container
```

Docker vs. Kubernetes

Docker — you directly start a container:

```
docker run nginx
```

Kubernetes — you declare intent, and Kubernetes creates and manages the Pods/containers for you:

```
Deployment
    |
    v
Pod
    |
    v
Nginx Container
```

A Kubernetes Deployment manages Pods and can restart them if necessary; Deployments are also used for scaling applications.

Part E — Download the Nginx Docker image

Step E1: Pull the Nginx image

We'll use Docker directly first, before bringing Kubernetes in.

Run:

```
docker pull nginx
```

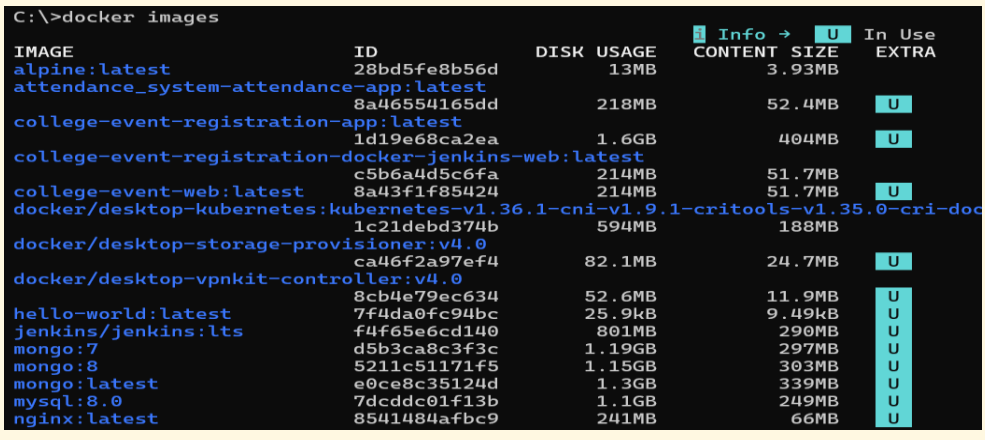
Step E2: Confirm the image was downloaded

Run:

```
docker images
```

Expected output:

```
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
nginx        latest    xxxxxxxxxxxx   ...           ...
```



Part F — Run Nginx using Docker

Step F1: Run the Nginx container

Understand the command before running it:

docker run — create a container

-d — run in the background

--name my-nginx — name the container my-nginx

-p 8080:80 — map computer port 8080 to container port 80

nginx — the Docker image to use

Run:

```
docker run -d --name my-nginx -p 8080:80 nginx
```

Step F2: Confirm the container is running

Run:

```
docker ps
```

Expected output:

```
... my-nginx ...
```

Step F3: View Nginx in the browser

Open your browser and go to the URL below. You should see the default Nginx welcome page.

Run:

```
http://localhost:8080
```

Expected output:

```
Welcome to nginx!
```

You have just deployed Nginx using Docker.

Part G — Stop the Docker container

We don't need this Docker container anymore because Kubernetes will run Nginx from here on.

Step G1: Stop the container

Run:

```
docker stop my-nginx
```

Step G2: Remove the container

Run:

```
docker rm my-nginx
```

Step G3: Confirm it's gone

Run:

```
docker ps
```

Expected output:

```
(my-nginx should no longer appear)
```

Part H — Deploy Nginx using Kubernetes

This is the important part of the practical.

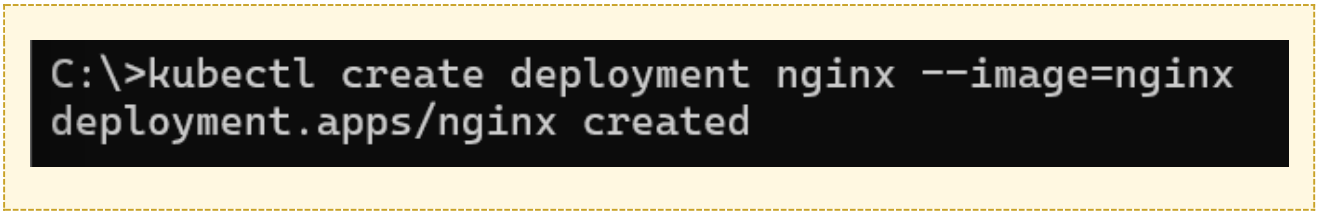
Step H1: Create the Deployment

Run:

```
kubectl create deployment nginx --image=nginx
```

Expected output:

```
deployment.apps/nginx created
```



You have now created your first Kubernetes Deployment.

Part I — Check the Deployment

Step I1: List Deployments

This means Kubernetes wants 1 Pod, and currently has 1 Pod running.

Run:

```
kubectl get deployments
```

Expected output:

```
NAME    READY   UP-TO-DATE   AVAILABLE
nginx   1/1     1            1
```

Part J — Check the Pod

Step J1: List Pods

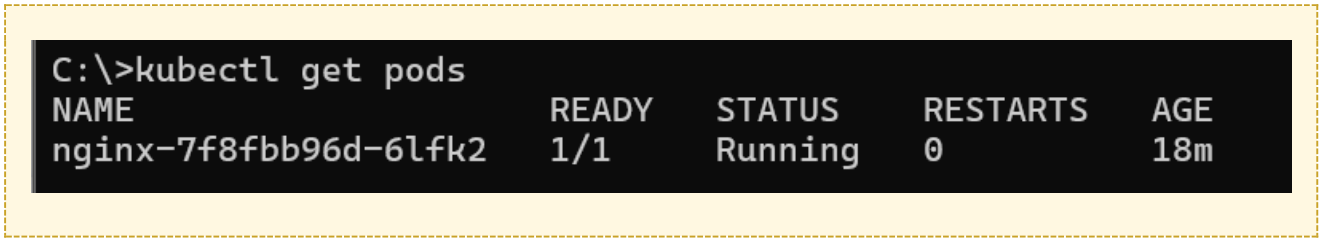
The random-looking name (e.g. nginx-7c79c4bf97-abcde) is normal — Kubernetes generated it. Don't worry about memorizing it.

Run:

```
kubectl get pods
```

Expected output:

```
NAME                     READY   STATUS    RESTARTS   AGE
nginx-xxxxxxxxxx-xxxxx   1/1     Running   0          30s
```



Part K — Understand Deployment and Pod

You now have the following chain of objects:

```
Kubernetes
    |
    v
Deployment: nginx
    |
    v
Pod
    |
    v
Nginx Container
    |
    v
Port 80
```

Object	Meaning
Deployment	Controls the desired number of Pods.
Pod	The basic execution unit in Kubernetes.
Container	Your actual application runs inside the container.

Part L — See the Pod details

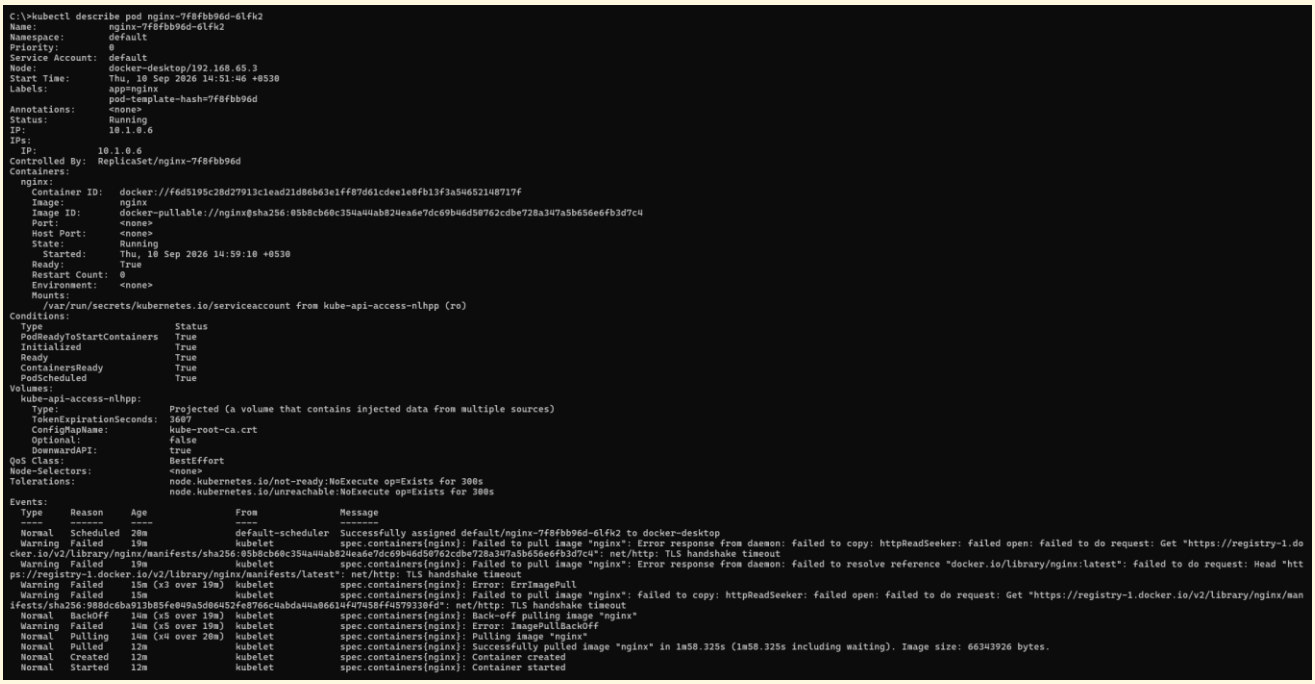
Step L1: Describe the Pod

Replace <POD_NAME> with the name you saw in Part J, e.g. nginx-7c79c4bf97-abcde. Look for the Containers: section in the output.

Run:

```
kubectl describe pod <POD_NAME>

# example:
kubectl describe pod nginx-7c79c4bf97-abcde
```



Step L2: Get extended Pod info

Run:

```
kubectl get pods -o wide
```

Part M — Check Nginx logs

Step M1: View Pod logs

Run:

```
kubectl logs <POD_NAME>

# example:
kubectl logs nginx-7c79c4bf97-abcde
```

This is the Kubernetes equivalent of the Docker command:

```
docker logs my-nginx     (Docker)
kubectl logs <POD_NAME>  (Kubernetes)
```

Part N — Our application is NOT accessible yet

You created Deployment → Pod → Nginx, but if you try http://localhost you should not expect your Kubernetes Nginx to automatically appear there.

Why? Because the Pod only has an internal Kubernetes network address. We need a Service to expose it.

Part O — Create a Kubernetes Service

Step O1: Expose the Deployment

Run:

```
kubectl expose deployment nginx --type=NodePort --port=80
```

Expected output:

```
service/nginx exposed
```

Step O2: List Services

The important part is the port mapping, e.g. 80:30080/TCP. Your exact NodePort will differ.

Run:

```
kubectl get services
```

Expected output:

```
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)
kubernetes   ClusterIP   10.x.x.x        <none>        443/TCP
nginx        NodePort    10.x.x.x        <none>        80:30xxx/TCP
```

```
C:\>kubectl get services
NAME         TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP        50m
nginx        NodePort    10.111.85.214   <none>        80:30885/TCP   11s
```

Part P — Access Nginx

Because you're using Docker Desktop Kubernetes, exactly how the Service is reachable can depend on your cluster setup.

Step P1: Check the Service

Run:

```
kubectl get service nginx
```

Step P2: Port-forward to the Service

This is the most reliable way to reach the Service from Docker Desktop. Keep this terminal running.

Run:

```
kubectl port-forward service/nginx 8080:80
```

Expected output:

```
Forwarding from 127.0.0.1:8080 -> 80
```

Step P3: Open Nginx in the browser

Run:

```
http://localhost:8080
```

Expected output:

```
Welcome to nginx!
```

You have now completed the full chain:

```
Docker Image
     |
     v
Kubernetes Deployment
     |
     v
Pod
     |
     v
Nginx Container
     |
     v
Kubernetes Service
     |
     v
Your Browser
```

Part Q — The most important experiment: scaling

Currently you have one Pod behind the Deployment:

```
nginx Deployment
       |
       v
     Pod 1
```

Step Q1: Scale the Deployment to 3 replicas

Run:

```
kubectl scale deployment nginx --replicas=3
```

Expected output:

```
deployment.apps/nginx scaled
```

Step Q2: Confirm 3 Pods are running

Run:

```
kubectl get pods
```

Expected output:

```
NAME                     READY   STATUS    RESTARTS
nginx-xxxxx-aaaaa        1/1     Running   0
nginx-xxxxx-bbbbb        1/1     Running   0
```

```
nginx-xxxxx-ccccc        1/1     Running   0
```

```
C:\>kubectl get pods
NAME                     READY   STATUS             RESTARTS   AGE
nginx-7f8fbb96d-6lfk2    1/1     Running            0          47m
nginx-7f8fbb96d-6z2rf    0/1     ImagePullBackOff   0          6m32s
nginx-7f8fbb96d-gsckk    0/1     ImagePullBackOff   0          6m20s
```

You now have:

```
             Deployment
                  |
      +-----------+-----------+
      |           |           |
      v           v           v
    Pod 1       Pod 2       Pod 3
      |           |           |
      v           v           v
   Nginx       Nginx       Nginx
```

This is one of the fundamental ideas behind Kubernetes.

Part R — Check the Deployment again

Step R1: List the Deployment

3/3 means three Pods are ready out of three desired Pods.

Run:

```
kubectl get deployment
```

Expected output:

```
NAME    READY   UP-TO-DATE   AVAILABLE
nginx   3/3     3            3
```

Part S — Test Kubernetes self-healing

Step S1: List Pods and copy one Pod name

Run:

```
kubectl get pods
```

Expected output:

```
(copy one Pod name, e.g. nginx-xxxxx-aaaaa)
```

Step S2: Delete that Pod

You might think your application is now down.

Run:

```
kubectl delete pod nginx-xxxxx-aaaaa
```

Step S3: Immediately check Pods again

You may see a new Pod already being created.

Run:

```
kubectl get pods
```

Step S4: Check again after a few seconds

You should eventually be back to 3 Pods. Because you told Kubernetes "I want 3 replicas", when it sees Desired = 3 but Actual = 2, it creates another Pod. This is the beginning of self-healing.

Run:

```
kubectl get pods
```

Expected output:

```
(3 Pods Running again)
```

```
C:\>kubectl get pods
NAME                     READY   STATUS             RESTARTS   AGE
nginx-7f8fbb96d-6lfk2    1/1     Running            0          49m
nginx-7f8fbb96d-6z2rf    0/1     ImagePullBackOff   0          7m56s
nginx-7f8fbb96d-gsckk    0/1     ImagePullBackOff   0          7m44s

C:\>kubectl delete pod nginx-7f8fbb96d-gsckk
pod "nginx-7f8fbb96d-gsckk" deleted from default namespace

C:\>kubectl get pods
NAME                     READY   STATUS              RESTARTS   AGE
nginx-7f8fbb96d-6lfk2    1/1     Running             0          50m
nginx-7f8fbb96d-6z2rf    0/1     ImagePullBackOff    0          9m
nginx-7f8fbb96d-bvsrw    0/1     ContainerCreating   0          13s
```

Part T — View everything together

Step T1: List all resources

This one command is very useful for beginners — it shows Pods, Services, Deployments, and ReplicaSets in one place.

Run:

```
kubectl get all
```

Expected output:

```
pod/...
pod/...
pod/...

service/nginx

deployment.apps/nginx

replicaset.apps/nginx
```

```
C:\>kubectl get all
NAME                         READY   STATUS             RESTARTS   AGE
pod/nginx-7f8fbb96d-6lfk2    1/1     Running            0          51m
pod/nginx-7f8fbb96d-6z2rf    0/1     ImagePullBackOff   0          10m
pod/nginx-7f8fbb96d-bvsrw    0/1     ErrImagePull       0          96s

NAME                 TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
service/kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP        75m
service/nginx        NodePort    10.111.85.214   <none>        80:30885/TCP   25m

NAME                    READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx   1/3     3            1           51m

NAME                               DESIRED   CURRENT   READY   AGE
replicaset.apps/nginx-7f8fbb96d    3         3         1       51m
```

Part U — Understand ReplicaSet

You didn't directly create a ReplicaSet — but when you created the Deployment, Kubernetes created one automatically.

```
Deployment
     |
     v
ReplicaSet
     |
     +---------+---------+
     |         |         |
     v         v         v
    Pod       Pod       Pod
```

Deployment manages the ReplicaSet, and the ReplicaSet manages the desired number of Pods.

Part V — Inspect the Service

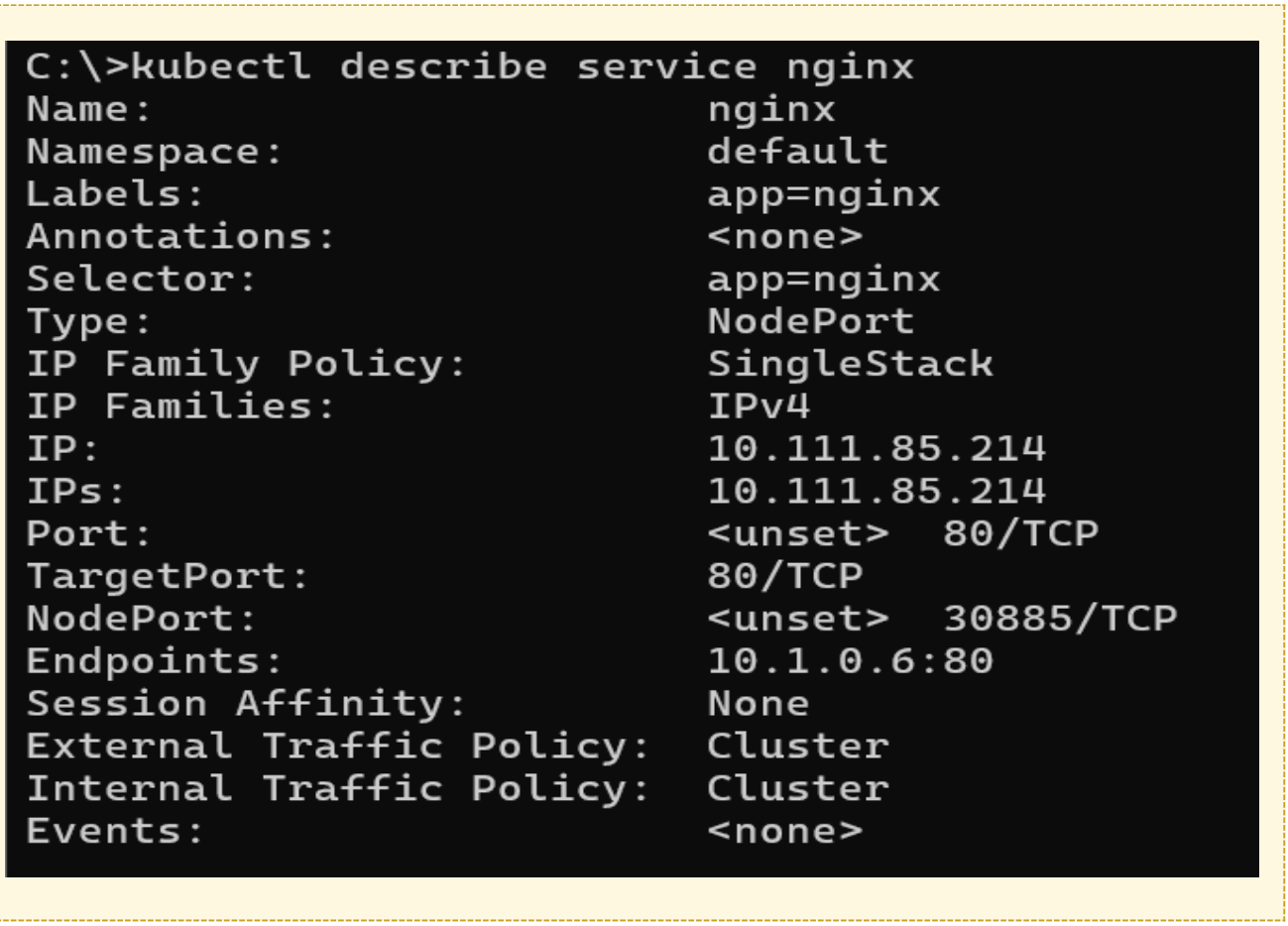
Step V1: Describe the Service

Run:

```
kubectl describe service nginx
```

Expected output:

```
Type: NodePort
Port: 80
TargetPort: 80
Endpoints: ...
```



Think of the traffic flow as:

```
Service
   |
```

```
   | port 80
   v
Pods
   |
   | port 80
   v
Nginx
```

Part W — Clean up

When your practical is complete, remove the resources you created.

Step W1: Delete the Service

Run:

```
kubectl delete service nginx
```

Step W2: Delete the Deployment

Run:

```
kubectl delete deployment nginx
```

Step W3: Confirm everything is gone

Run:

```
kubectl get all
```

Expected output:

```
(your nginx resources should no longer appear)
```

```
C:\>kubectl delete service nginx
service "nginx" deleted from default namespace

C:\>kubectl delete deployment nginx
deployment.apps "nginx" deleted from default namespace

C:\>kubectl get all
NAME                     READY   STATUS        RESTARTS   AGE
pod/nginx-7f8fbb96d-bvsrw   0/1     Terminating   0          4m

NAME                 TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
service/kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   78m
```

What you should understand after Practical 1

Concept	Simple meaning
Docker Image	Blueprint for a container
Container	Running instance of an image
Kubernetes Cluster	Kubernetes environment
Node	Machine running Kubernetes workloads
Pod	Smallest deployable Kubernetes unit
Deployment	Manages application Pods
ReplicaSet	Maintains desired number of Pods
Service	Provides stable network access to Pods
NodePort	Exposes a Service through a node port
Scaling	Increasing/decreasing number of Pods
Self-healing	Kubernetes recreates failed/deleted Pods

Appendix — Full command list (lab notebook copy)

```
docker --version
docker pull nginx
docker run -d --name my-nginx -p 8080:80 nginx
docker ps
docker stop my-nginx
docker rm my-nginx
kubectl get nodes
kubectl create deployment nginx --image=nginx
kubectl get deployments
kubectl get pods
kubectl get pods -o wide
kubectl describe pod <POD_NAME>
kubectl logs <POD_NAME>
kubectl expose deployment nginx --type=NodePort --port=80
kubectl get services
kubectl port-forward service/nginx 8080:80
kubectl scale deployment nginx --replicas=3
kubectl get pods
kubectl get deployment
kubectl delete pod <POD_NAME>
kubectl get pods
kubectl get all
kubectl delete service nginx
kubectl delete deployment nginx
```

Submission checklist

- ☐ Screenshot: docker ps output (Part A)
- ☐ Screenshot: kubectl get nodes showing Ready (Part C)
- ☐ Screenshot: Docker Desktop Kubernetes settings (Part B)
- ☐ Screenshot: docker images showing nginx:latest (Part E)
- ☐ Screenshot: browser showing Nginx via Docker, localhost:8080 (Part F)
- ☐ Screenshot: kubectl create deployment confirmation (Part H)
- ☐ Screenshot: kubectl get pods showing Running (Part J)
- ☐ Screenshot: kubectl describe pod output (Part L)
- ☐ Screenshot: kubectl get services showing NodePort (Part O)
- ☐ Screenshot: browser showing Nginx via Kubernetes Service (Part P)
- ☐ Screenshot: kubectl get pods showing 3 replicas (Part Q)
- ☐ Screenshot: self-healing before/after (Part S)
- ☐ Screenshot: kubectl get all (Part T)
- ☐ Screenshot: kubectl describe service nginx (Part V)
- ☐ Screenshot: final kubectl get all after cleanup (Part W)
- ☐ Short reflection: in your own words, explain the difference between a Deployment, a Pod, and a Service.